Cryptocurrency Liquidity Prediction

Systematic project solution using the uploaded CoinGecko CSV files

Project	Cryptocurrency Liquidity Prediction for Market Stability
Source files used	coin_gecko_2022-03-16.csv and coin_gecko_2022-03-17.csv
Rows used	993 valid rows after target filtering (1000 combined rows before filtering)
Unique coins	503
Date coverage in uploaded data	2022-03-16 to 2022-03-17

Important note: the assignment brief mentions historical records from 2016 and 2017, but the files attached with the request contain March 2022 CoinGecko snapshots. This solution therefore follows the project instructions while using the attached 2022 CSV files exactly as provided.

1. Problem Understanding

The uploaded project brief asks for a machine learning workflow that predicts cryptocurrency liquidity, including preprocessing, exploratory analysis, feature engineering, model training, evaluation, tuning, testing, and a simple deployment concept. The main deliverables requested are a trained model, evaluation metrics, an EDA report, HLD, LLD, pipeline explanation, and a final report.

Business Objective

Liquidity reflects how easily a cryptocurrency can be traded without causing major price disruption. Low liquidity can make markets unstable and can expose traders and platforms to risk. The goal of this solution is to estimate a liquidity indicator from available market features so that unusual low-liquidity conditions can be identified earlier.

2. Dataset Overview

The two uploaded CSV files were combined into a single dataset with 1000 rows and 14 original columns. After removing rows where the target could not be computed, 993 rows remained for modeling. The dataset contains 503 unique coins across two collection dates.

Column	Missing values before preprocessing
1h	7
24h	7
7d	8
24h_volume	7

Core fields used

coin, symbol, price, 1h return, 24h return, 7d return, 24h trading volume, market capitalization, and date.

3. Data Preprocessing

The preprocessing steps were organized in the following order: combine both CSV files, convert date to datetime, convert numeric columns safely, replace infinite values with missing values, create the liquidity target, engineer volatility and change features, and finally impute remaining missing values with the median during model training. Median imputation was chosen because financial variables can be highly skewed.

Target definition

Since the attached files do not contain a direct liquidity label, liquidity was approximated using the liquidity ratio:

```
liquidity_ratio = 24h_volume / mkt_cap
```

This ratio is a practical proxy for turnover. Higher values indicate more trading activity relative to market size, which generally suggests stronger liquidity.

4. Exploratory Data Analysis (EDA)

The EDA focused on understanding price distribution, the relationship between market capitalization and trading volume, correlation among numeric variables, and the strongest and weakest liquidity cases in the uploaded data.

Metric	Price	24h Volume	Market Cap	Liquidity Ratio
mean	619.4355	289310894.1975	3780940696.2779	0.1037
50%	1.1000	8343005.0000	213564555.0000	0.0338
min	0.0000	0.0000	65770433.0000	0.0000
max	41217.2700	57934969077.0000	776077432316.0000	5.9485

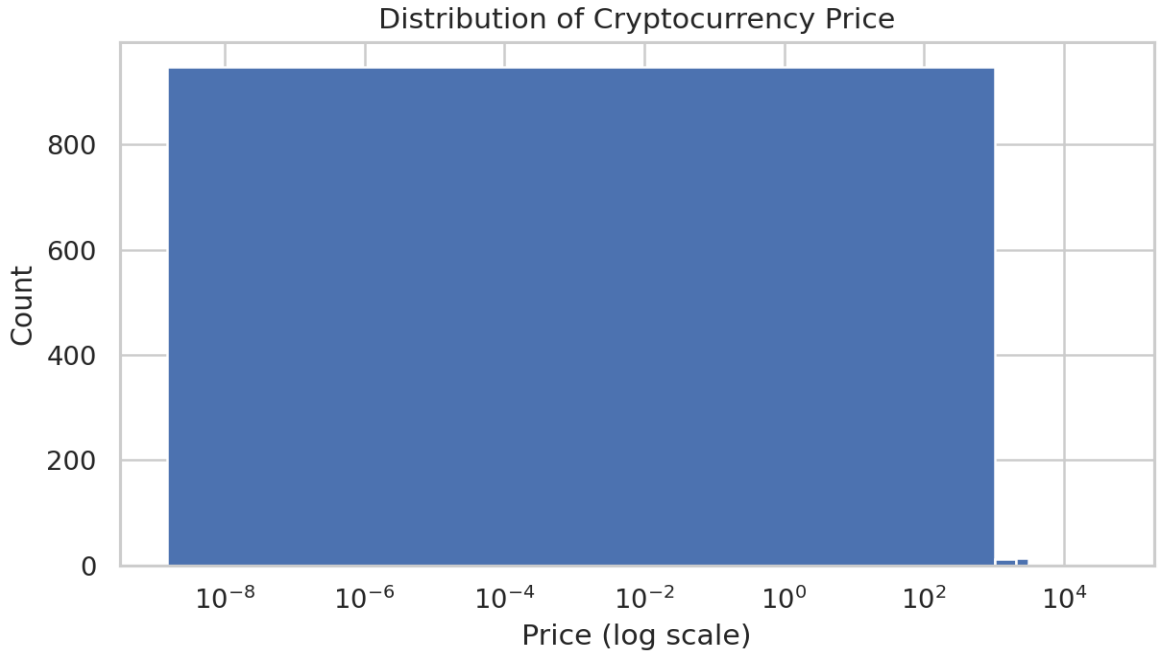
Main observations from the uploaded data

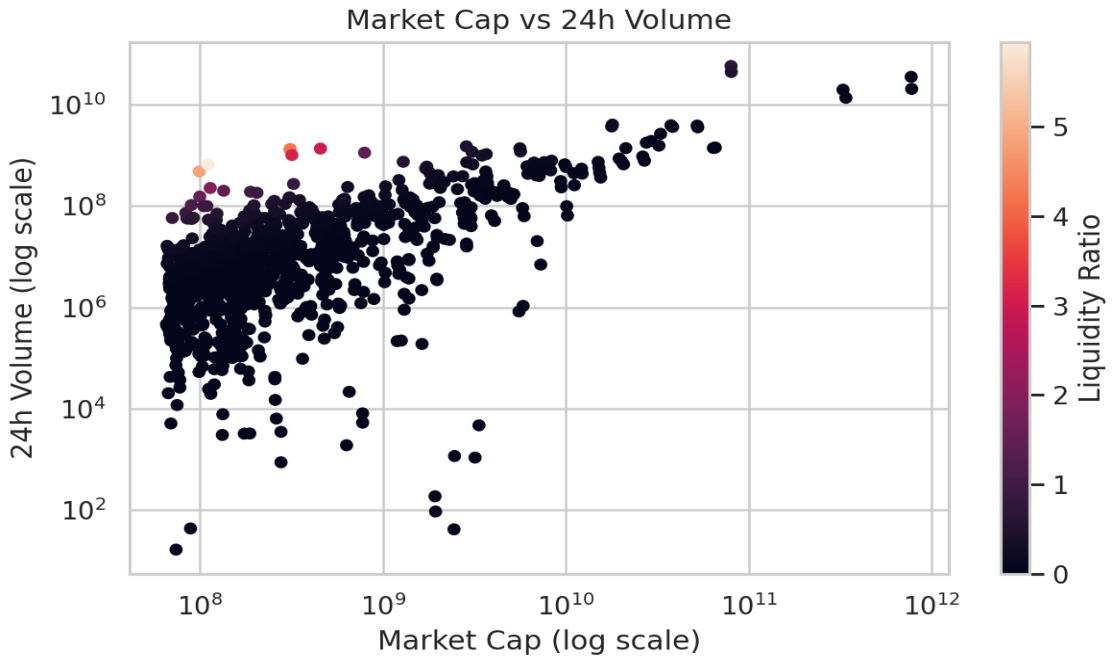
- The dataset is highly skewed. The median liquidity ratio is 0.0338, while the maximum reaches 5.9485.

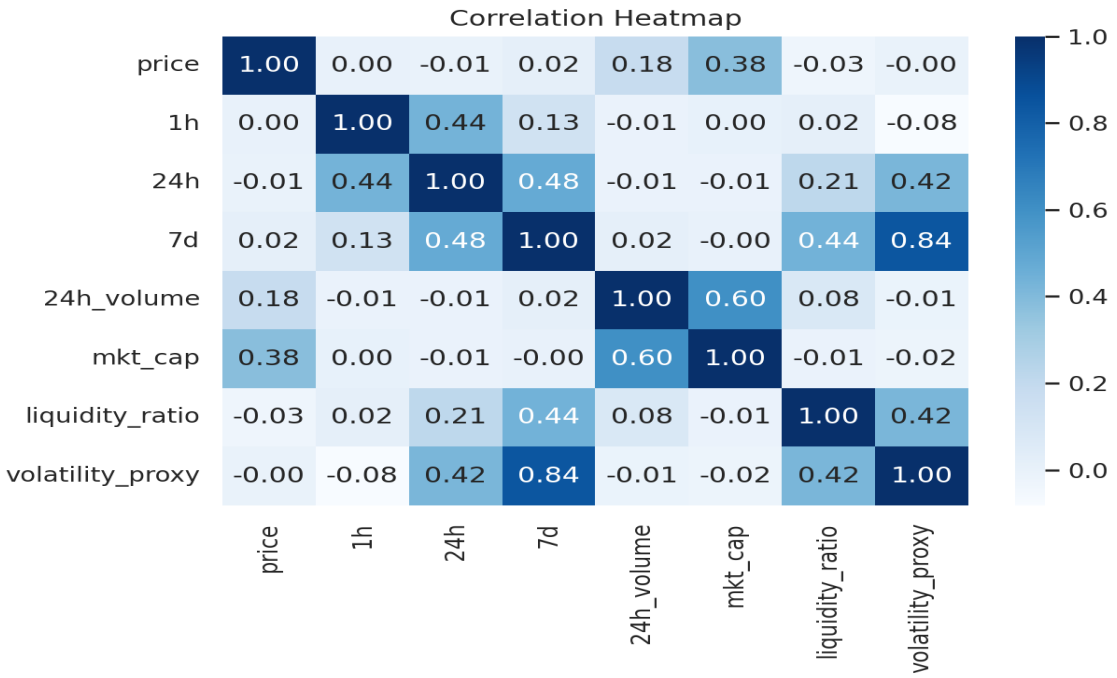
- The strongest positive simple correlation with liquidity ratio is the 7-day return (0.4406), followed by the volatility proxy (0.4179).

- High-liquidity coins in the uploaded snapshot include JasmyCoin, FLEX Coin, and STEPN, while extremely low-liquidity observations include Pundi X [OLD], cUSDC, and cDAI.

- Market capitalization and 24h volume dominate model-based feature importance, which is reasonable because liquidity ratio is built from them and reflects turnover behavior.



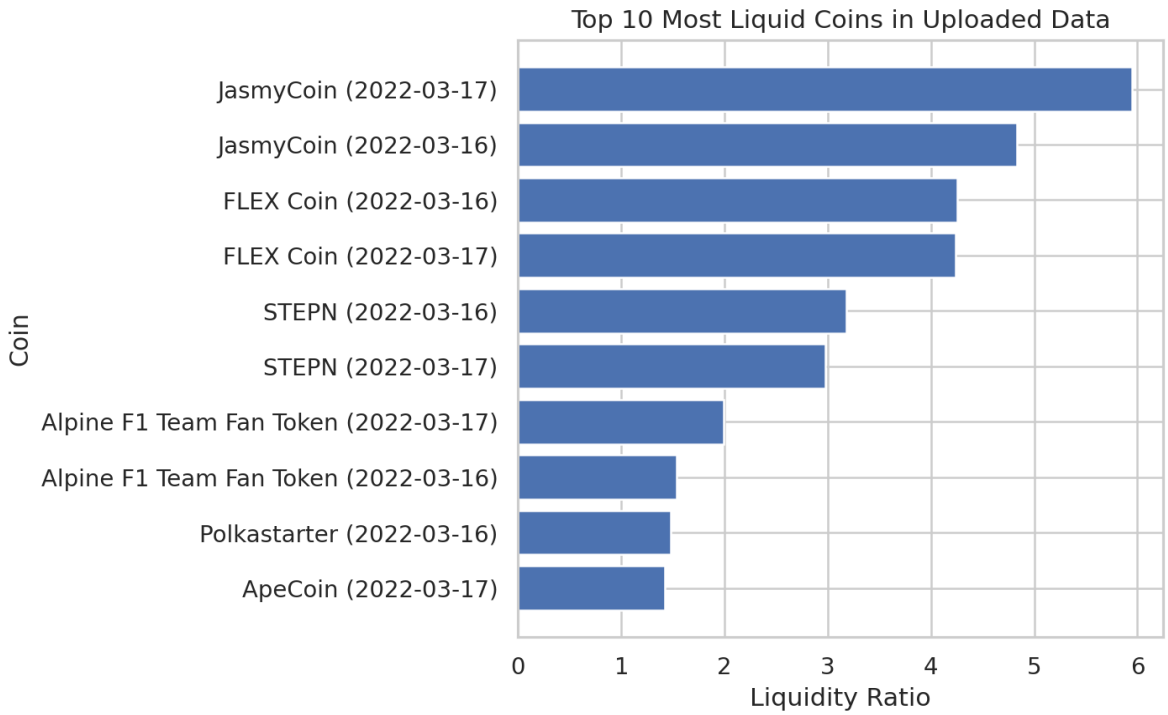



Correlation Heatmap
price 1.00 0.00 -0.01 0.02 0.18 0.38 -0.03 -0.00
1h 0.00 1.00 0.44 0.13 -0.01 0.00 0.02 -0.08
24h -0.01 0.44 1.00 0.48 -0.01 -0.01 0.21 0.42
7d 0.02 0.13 0.48 1.00 0.02 -0.00 0.44 0.84
24h_volume 0.18 -0.01 -0.01 0.02 1.00 0.60 0.08 -0.01
mkt_cap 0.38 0.00 -0.01 -0.00 0.60 1.00 -0.01 -0.02
liquidity_ratio -0.03 0.02 0.21 0.44 0.08 -0.01 1.00 0.42
volatility_proxy -0.00 -0.08 0.42 0.84 -0.01 -0.02 0.42 1.00
price
1h
24h
7d
24h_volume
mkt_cap
liquidity_ratio
volatility_proxy
1.0
0.8
0.6
0.4
0.2
0.0

5. Feature Engineering

To follow the assignment requirement for engineered liquidity-related features, the following new variables were created:

Feature	Definition	Purpose
liquidity_ratio	24h_volume / mkt_cap	Target variable for liquidity prediction
volatility_proxy	mean(abs(1h), abs(24h), abs(7d))	Captures short-term return instability
price_change_day	pct_change(price) by coin	Measures daily price movement per coin
volume_change_day	pct_change(24h_volume) by coin	Measures daily turnover shift per coin
is_second_day	1 if date is latest date else 0	Simple indicator for snapshot timing

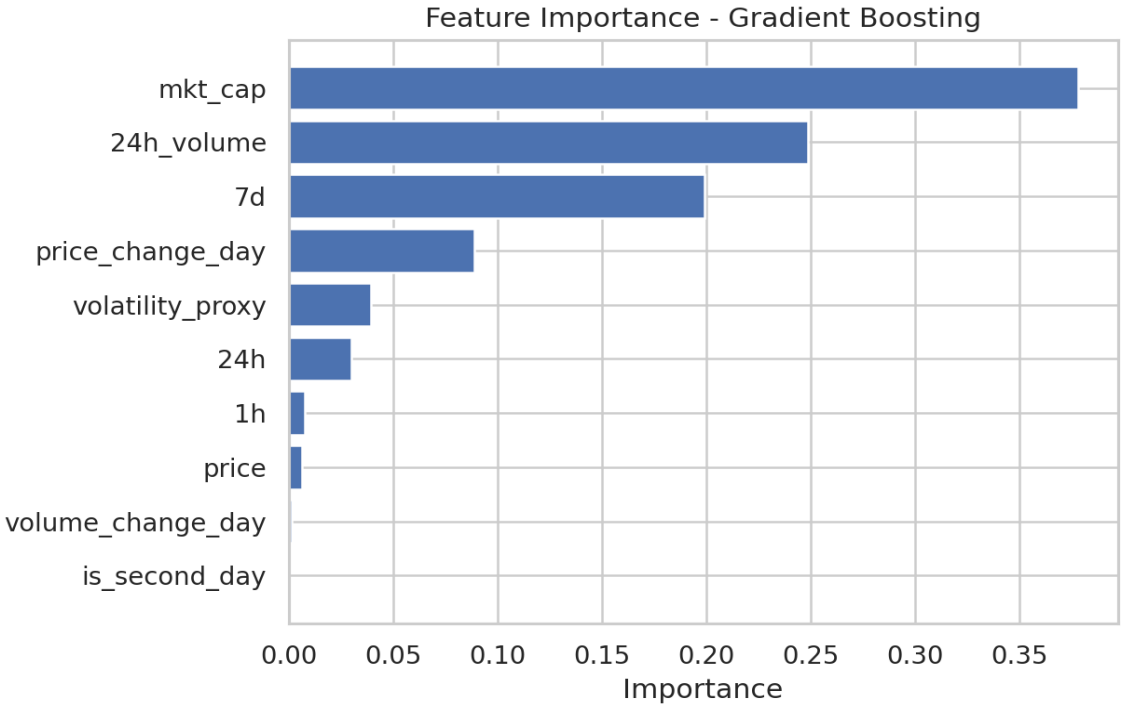


6. Model Selection and Training

Because the target is continuous, the task was treated as a regression problem. Three models were compared: Linear Regression as a baseline, Random Forest Regressor as a non-linear ensemble benchmark, and Gradient Boosting Regressor as the final selected model. The data was split into 80 percent training and 20 percent testing.

Model	MAE	RMSE	R2
Linear Regression	0.1622	0.6302	0.0746
Random Forest	0.0729	0.428	0.5732
Gradient Boosting	0.0762	0.42	0.589

Gradient Boosting achieved the best overall result on the uploaded data with the lowest RMSE and the highest R2, so it was selected as the final model for this report.



7. High-Level Design (HLD)

Data Source -> Preprocessing -> Feature Engineering -> EDA -> Train/Test Split -> Model Training -> Evaluation -> Saved Model / Dashboard Input

At the high level, the system begins with the uploaded CoinGecko files, transforms them into a cleaned analytics table, trains a regression model, and then exposes predictions for reporting or lightweight local deployment.

8. Low-Level Design (LLD)

Module 1: Data Loader - reads both CSV files and combines them. Module 2: Data Cleaner - converts numeric columns, parses dates, handles missing and infinite values. Module 3: Feature Engineering - creates liquidity_ratio, volatility_proxy, and daily change features. Module 4: EDA Module - generates summary statistics and plots. Module 5: Model Trainer - trains baseline and ensemble regressors. Module 6: Evaluator - computes MAE, RMSE, and R2. Module 7: Inference Layer - accepts new market inputs and returns predicted liquidity.

9. Pipeline Architecture

Step 1: Read attached CSV files. Step 2: Clean and standardize the data. Step 3: Build liquidity-related features. Step 4: Generate EDA outputs and validate assumptions. Step 5: Train candidate models. Step 6: Evaluate models on the holdout set. Step 7: Select the best model and save the code pipeline. Step 8: Use a Streamlit or Flask front end for local testing.

10. Local Deployment Idea

A simple Streamlit app can collect price, returns, volume, and market capitalization for a coin, pass them through the same preprocessing logic, and display the predicted liquidity ratio. This would allow analysts to test market scenarios interactively.

11. Final Conclusion

This solution follows the assignment workflow systematically and uses the attached data rather than a generic example dataset. The uploaded CoinGecko files show that liquidity in this snapshot is highly skewed, and the strongest model in this report is Gradient Boosting Regressor. The most influential predictors are market capitalization, 24h trading volume, and 7-day return behavior. Even though the uploaded data only spans two dates, the project still demonstrates the full machine learning pipeline requested in the assignment brief.

12. Reusable Python Code

The full code is also attached separately as a .py file. A shortened core version is shown below.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load uploaded CoinGecko files
df1 = pd.read_csv("coin_gecko_2022-03-16.csv")
df2 = pd.read_csv("coin_gecko_2022-03-17.csv")
df = pd.concat([df1, df2], ignore_index=True)

# Convert columns
numeric_cols = ["price", "1h", "24h", "7d", "24h_volume", "mkt_cap"]
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df["date"] = pd.to_datetime(df["date"])
df = df.sort_values(["coin", "date"]).copy()

# Feature engineering
df["liquidity_ratio"] = df["24h_volume"] / df["mkt_cap"]
df["volatility_proxy"] = df[["1h", "24h", "7d"]].abs().mean(axis=1)
df["price_change_day"] = df.groupby("coin")["price"].pct_change(fill_method=None)
df["volume_change_day"] = df.groupby("coin")["24h_volume"].pct_change(fill_method=None)
df["is_second_day"] = (df["date"] == df["date"].max()).astype(int)
df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Keep rows with valid target
df = df[df["liquidity_ratio"].notna()].copy()

# Features and target
features = [
    "price", "1h", "24h", "7d", "24h_volume", "mkt_cap",
    "volatility_proxy", "price_change_day", "volume_change_day", "is_second_day"
]
X = df[features]
y = df["liquidity_ratio"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Impute missing values
imputer = SimpleImputer(strategy="median")
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)

# Train final model
model = GradientBoostingRegressor(
    n_estimators=200, learning_rate=0.05, max_depth=2, subsample=0.8, random_state=42
)
model.fit(X_train_imp, y_train)

# Predict
pred = model.predict(X_test_imp)
```

```
# Evaluate
mae = mean_absolute_error(y_test, pred)
rmse = mean_squared_error(y_test, pred) ** 0.5
r2 = r2_score(y_test, pred)

print("MAE:", round(mae, 4))
print("RMSE:", round(rmse, 4))
print("R2:", round(r2, 4))

# Feature importance
importance = pd.Series(model.feature_importances_, index=features).sort_values(ascending=False)
print("\nFeature importance:")
print(importance)
```